

STUDENT MANAGEMENT SYSTEM

Console-Based Java Application

Project Documentation

Language	Java (JDK 8+)
Paradigm	Object-Oriented Programming
Interface	Command-Line (Scanner-based)
Core Classes	Student, StudentManagementSystem
Data Storage	In-memory (ArrayList<Student>)
Document Type	Technical & Functional Documentation

Table of Contents

1. Introduction	3
2. Project Objectives	4
3. Scope of the System	5
4. System Requirements	6
5. System Architecture & Design	7
6. Class Documentation — Student	10
7. Class Documentation — StudentManagementSystem	14
8. Method-by-Method Walkthrough	16
9. Menu Flow & Control Logic	28
10. Sample Execution Transcripts	30
11. Test Cases & Validation	36
12. Grading & Attendance Policy Reference	40
13. Known Limitations	41
14. Suggested Enhancements	43
15. Advantages of the System	45
16. Conclusion	46
17. Viva-Voce Question Bank	47
18. Appendix A — Complete Source Code	50
19. Appendix B — References	58

Note: page numbers above are indicative section markers within this document.

1. Introduction

The Student Management System is a console-based Java application designed to help an educational institution, department, or individual instructor keep track of core student records without relying on a database server or external storage. The program is intentionally lightweight: it runs entirely in memory, uses the standard `java.util.Scanner` class for input, and `java.util.ArrayList` for storage, making it an ideal learning project for students who are studying object-oriented programming, control flow, and basic data structures in Java.

At its core, the application models a single real-world entity — a Student — as a Java class with six attributes: registration number, name, branch, marks, total classes conducted, and classes attended. Around this model, a second class, `StudentManagementSystem`, implements a menu-driven command-line interface that lets an authenticated user add, view, search, update, delete, and analyse student records. The analysis functions include automatic grade computation from marks and automatic attendance-percentage computation with an eligibility check.

This documentation describes the system from four complementary angles. First, it explains what the program is meant to do and why it is structured the way it is (Sections 1–5). Second, it walks through every class and method in detail, including the exact logic used for grading and attendance (Sections 6–9). Third, it demonstrates the program in action with realistic sample transcripts and a structured set of test cases (Sections 10–12). Finally, it closes with a candid discussion of the system's limitations, ideas for extending it, and a viva-voce question bank that mirrors the kind of questions typically asked when this style of mini-project is presented for academic evaluation (Sections 13–17), followed by the complete, unmodified source code as an appendix.

The intended audience for this document includes the student or developer who wrote the program and needs a companion report, an evaluator or reviewer who wants to understand the program's behaviour without reading raw code line by line, and any future maintainer who might extend the project — for example, by adding file or database persistence, a graphical interface, or multi-user support.

2. Project Objectives

The Student Management System was built with a small, focused set of objectives in mind. These objectives double as the acceptance criteria against which the finished program can be checked.

- Provide a single, self-contained Java program that can record essential student information (registration number, name, branch, marks, and attendance figures) without requiring a database.
- Demonstrate core object-oriented programming concepts — encapsulation of data inside a Student class, methods that operate on that data, and a separate controller class that orchestrates the menu-driven workflow.
- Offer complete CRUD (Create, Read, Update, Delete) functionality for student records through a numbered console menu that is easy to navigate for a non-technical end user.
- Automatically derive two pieces of academic insight from the raw data the user enters: a letter grade from the marks, and an attendance percentage with an eligibility verdict from the attended/total class counts.
- Guard access to the system with a simple login step so that the menu is not reachable without a valid username and password.
- Keep the codebase small enough (under 300 lines) that a student can read, understand, explain, and modify every line of it during a viva or code review.
- Serve as a stepping stone toward more advanced versions of the same idea — for instance, a version backed by a file, a relational database, or a graphical user interface.

3. Scope of the System

Understanding what the system does — and just as importantly, what it deliberately does not do — is essential before diving into its design. The scope below reflects the program exactly as written, without assuming features that would require additional code.

3.1 In Scope

- Adding a new student record with registration number, name, branch, marks, total classes, and attended classes.
- Displaying every stored student record, including derived grade and attendance percentage.
- Searching for a single student record by registration number.
- Updating a student's name, branch, and marks by registration number.
- Deleting a student record by registration number.
- Calculating and displaying a letter grade (A–F) from a student's marks.
- Calculating and displaying a percentage figure derived directly from the marks field.
- Calculating attendance percentage and an eligibility status (Eligible / Not Eligible) using a 75% threshold.
- A one-time login gate at program start-up using a hard-coded username and password.

3.2 Out of Scope

- Persistent storage: all data lives only in memory (an ArrayList) and is lost the moment the program exits, because nothing is written to a file or database.
- Multi-user support or role-based access: the login check accepts exactly one hard-coded username/password pair and does not distinguish between an administrator and a student.
- Input validation: the program does not check that marks fall within 0–100, that attended classes do not exceed total classes, or that a registration number is not already in use.
- A graphical user interface: all interaction happens through `System.out.println` and `Scanner` in a terminal window.
- Networking or client-server behaviour: the program is a single-process, single-machine console application.
- Reporting or export features such as printing a class list to a file, generating a PDF report card, or exporting to a spreadsheet.

4. System Requirements

4.1 Software Requirements

Component	Requirement
Programming Language	Java (source and target compatible with JDK 8 or later)
Compiler / Runtime	javac and java from a standard JDK installation
Operating System	Any OS with a JVM: Windows, macOS, or Linux
IDE (optional)	Any text editor or IDE, e.g. IntelliJ IDEA, Eclipse, VS Code, or a plain terminal
External Libraries	None — only java.util.ArrayList and java.util.Scanner from the standard library

4.2 Hardware Requirements

Resource	Minimum
Processor	Any modern processor capable of running a JVM
RAM	512 MB or more (the program itself uses a negligible amount of memory)
Disk Space	A few megabytes for the JDK installation and compiled class files
Display	A standard terminal / console window

4.3 Build & Run Instructions

The program is a single source file named `StudentManagementSystem.java` containing two classes: the package-private `Student` class and the public `StudentManagementSystem` class that holds the entry point. It can be compiled and executed from a terminal as shown below.

```
javac StudentManagementSystem.java
java StudentManagementSystem
```

Because the class is public, the file name must exactly match the public class name (`StudentManagementSystem.java`). No command-line arguments are required; the program immediately prompts for a username and password when it starts.

5. System Architecture & Design

The application follows a very small two-class architecture that separates data (the Student class) from behaviour and orchestration (the StudentManagementSystem class). This is a lightweight, informal approximation of the Model – Controller pattern: Student is the model, and StudentManagementSystem plays the role of both controller and view, since it also owns all of the console input/output.

5.1 High-Level Component View

Layer	Responsibility	Implemented By
Data Model	Represents one student and derives grade/attendance from raw data	Student class
Storage	Holds the in-memory collection of all students for the running session	static ArrayList<Student> students
Input	Reads console input for every menu operation	static Scanner sc
Business Logic	Add/search/update/delete/grade/attendance operations	static methods in StudentManagementSystem
Presentation	Prints menus, prompts, and results to the console	System.out.println calls throughout
Control Flow	Login gate, then a do-while loop driving a switch-based menu	main() method

5.2 Class Relationship

StudentManagementSystem has a one-to-many relationship with Student, implemented as an ArrayList<Student> field named students. Every operation exposed by the menu (add, search, update, delete, grade, percentage, attendance) is a static method on StudentManagementSystem that reads from or writes to this single shared list. Student itself has no reference back to StudentManagementSystem, so the dependency is one-directional.

A simplified textual class diagram is shown below, listing each class's fields and methods together with their visibility as declared in the source file.

```
+-----+
|          Student          |
+-----+
| + regNo : int             |
| + name : String           |
| + branch : String         |
| + marks : int             |
| + totalClasses : int      |
| + attendedClasses : int   |
+-----+
| + Student(regNo, name, branch, |
|           marks, total, attended) |
| + getAttendance() : double |
| + getGrade() : char       |
| + display() : void        |
+-----+

+-----+
| StudentManagementSystem |
+-----+
| ~ students : ArrayList<Student> |
| ~ sc : Scanner                 |
+-----+
| + addStudent() : void          |
| + displayStudents() : void     |
| + searchStudent() : void      |
| + updateStudent() : void      |
| + deleteStudent() : void      |
| + calculateGrade() : void     |
| + calculatePercentage() : void |
| + login() : boolean           |
| + attendance() : void         |
| + main(String[]) : void       |
+-----+
```

5.3 Data Flow

Data flows in a simple cycle for every menu choice: the user types a number at the "Enter your choice:" prompt, the switch statement in main() dispatches to the matching static method, that

method reads any additional input it needs directly from the shared Scanner, it then reads or mutates the shared students list, and finally it prints a result or confirmation message before control returns to the do-while loop, which redraws the menu and waits for the next choice.

5.4 Design Rationale

- Static fields and methods: because the program has only one "instance" of the whole system for the life of the process, everything is declared static rather than wrapping the menu logic in an instantiated object.
- Single shared Scanner: reusing one Scanner object avoids the classic bug of opening multiple Scanners over System.in, which can cause input to be consumed unexpectedly.
- ArrayList over array: an ArrayList is used instead of a fixed-size array so the number of students is not capped at compile time and deletion does not require manual shifting of elements.
- Derived fields kept as methods, not stored fields: grade and attendance percentage are computed on demand from `marks / attendedClasses / totalClasses` rather than being stored, which guarantees they are always consistent with the latest values even after an update.

6. Class Documentation — Student

The Student class is a plain, package-private data-holder class. It has no getters/setters in the conventional sense (fields are accessed directly since the class is used only within the same file), a single constructor, and three behavioural methods.

6.1 Fields

Field	Type	Description
regNo	int	Unique registration number used as the key for search, update, and delete operations.
name	String	Student's full name (read as a single token via <code>Scanner.next()</code>).
branch	String	Student's department or branch of study (also a single token).
marks	int	Marks obtained, assumed to be on a 0–100 scale, used to derive the letter grade and the percentage.
totalClasses	int	Total number of classes conducted, used as the denominator for the attendance calculation.
attendedClasses	int	Number of classes the student attended, used as the numerator for the attendance calculation.

6.2 Constructor

```
Student(int regNo, String name, String branch, int marks,
        int totalClasses, int attendedClasses) {
    this.regNo = regNo;
    this.name = name;
    this.branch = branch;
    this.marks = marks;
    this.totalClasses = totalClasses;
    this.attendedClasses = attendedClasses;
}
```

The constructor takes all six field values as parameters and assigns them directly with no validation — for instance, it will happily accept a negative mark or an `attendedClasses` value greater than `totalClasses`. Every field is set exactly once, at construction time, which means the only way to change name, branch, or marks afterwards is through `updateStudent()` in the controller class (`regNo`, `totalClasses`, and `attendedClasses` are not editable through any menu option once a record is created).

6.3 getAttendance()

```
double getAttendance() {
    return ((double) attendedClasses / totalClasses) * 100;
}
```

This method returns the attendance percentage as a double. The cast to `(double)` on `attendedClasses` is essential: without it, Java would perform integer division and truncate the result, so, for example, 18 attended out of 20 total would evaluate to 0 instead of 90.0. Because there is no zero-check on `totalClasses`, calling this method on a record where `totalClasses` is 0 will produce a division-by-zero condition; for floating-point division in Java this does not throw an exception but instead yields Infinity or NaN, which will then be printed as-is by `display()` or `attendance()`.

6.4 getGrade()

```

char getGrade() {
    if (marks >= 90)
        return 'A';
    else if (marks >= 80)
        return 'B';
    else if (marks >= 70)
        return 'C';
    else if (marks >= 60)
        return 'D';
    else
        return 'F';
}

```

`getGrade()` implements a simple threshold ladder that maps the `marks` field onto one of five letter grades. The comparisons are evaluated top-down, so a mark of 95 matches the first branch (`marks >= 90`) and immediately returns 'A' without checking the remaining conditions. Any mark below 60, including negative numbers (since there is no lower-bound check), falls through every branch and returns 'F'.

6.5 display()

```

void display() {
    System.out.println("Reg No      : " + regNo);
    System.out.println("Name       : " + name);
    System.out.println("Branch    : " + branch);
    System.out.println("Marks     : " + marks);
    System.out.println("Grade     : " + getGrade());
    System.out.println("Attendance : " + getAttendance() + "%");
    System.out.println("-----");
}

```

`display()` prints every stored field plus the two derived values (grade and attendance) in a fixed, human-readable layout, followed by a dashed separator line. It is called both from `displayStudents()` (once per student, to list the whole roster) and from `searchStudent()` (once, for the single matching record). Because it calls `getGrade()` and `getAttendance()` itself, the derived values shown are always computed fresh at print time.

7. Class Documentation — StudentManagementSystem

StudentManagementSystem is the public class containing the program's entry point (main) along with every menu operation. All of its members are static, so the class is never instantiated; it behaves like a namespace holding one shared list of students, one shared Scanner, and ten operations.

7.1 Static Fields

Field	Type	Purpose
students	ArrayList<Student>	The single in-memory collection of all student records created during this run of the program.
sc	Scanner	A single Scanner wrapping System.in, shared by every method that needs console input.

7.2 Method Inventory

#	Method	Return Type	Purpose
1	addStudent()	void	Reads six values from the console and appends a new Student to the list.
2	displayStudents()	void	Prints every student currently stored, or a "No Students Available!" message.
3	searchStudent()	void	Finds and prints one student by registration number.
4	updateStudent()	void	Edits an existing student's name, branch, and marks.
5	deleteStudent()	void	Removes a student record by registration number.
6	calculateGrade()	void	Prints a student's marks and derived letter grade.
7	calculatePercentage()	void	Prints a student's marks as a percentage figure.
8	login()	boolean	Prompts for a username/password and validates them against a hard-coded pair.
9	attendance()	void	Prints attendance percentage and an eligibility verdict.
10	main(String[])	void	Entry point: calls login(), then runs the menu loop.

8. Method-by-Method Walkthrough

This section documents each StudentManagementSystem method in detail: its signature, the exact code, a plain-language explanation of its logic, and the edge cases a reader should be aware of.

8.1 addStudent()

```
static void addStudent() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    System.out.print("Enter Student Name: ");
    String name = sc.next();
    System.out.print("Enter Branch: ");
    String branch = sc.next();
    System.out.print("Enter Marks: ");
    int marks = sc.nextInt();
    System.out.print("Enter Total Classes: ");
    int total = sc.nextInt();
    System.out.print("Enter Attended Classes: ");
    int attended = sc.nextInt();
    students.add(new Student(
        regNo, name, branch, marks, total, attended));
    System.out.println("Student Added Successfully!");
}
```

addStudent() collects the six pieces of information needed to construct a Student in a fixed order: registration number, name, branch, marks, total classes, and attended classes. Each value is read with the Scanner method appropriate to its type — nextInt() for the four integer fields and next() for the two String fields. Once all six values have been read, a new Student object is constructed and appended to the shared students list with ArrayList.add(), and a confirmation message is printed.

Edge cases / things to watch for:

- sc.next() reads a single whitespace-delimited token, so a name or branch containing a space (e.g. "Mary Ann") will be truncated to just the first word.
- There is no check that the entered regNo is not already used by another record, so duplicate registration numbers are allowed and will make search/update/delete operate on only the first match found.
- None of the numeric fields are range-checked, so a negative mark or an attendedClasses value larger than totalClasses will be accepted silently.

8.2 displayStudents()

```
static void displayStudents() {
    if (students.isEmpty()) {
        System.out.println("No Students Available!");
        return;
    }
    System.out.println("\n===== STUDENT DETAILS =====");
    for (Student s : students) {
        s.display();
    }
}
```

displayStudents() first guards against an empty list with ArrayList.isEmpty(); if there are no records yet, it prints a friendly message and returns immediately. Otherwise it prints a header line and then

iterates over every element of students with an enhanced for-loop, calling each student's own `display()` method in turn. This keeps the formatting logic in one place (inside `Student.display()`) rather than duplicating it here.

Edge cases / things to watch for:

- The list is displayed in insertion order, not sorted by registration number, name, or any other field.
- Because `display()` calls `getAttendance()` and `getGrade()` internally, any record with `totalClasses` equal to zero will show Infinity or NaN in the Attendance line rather than raising a visible error.

8.3 searchStudent()

```
static void searchStudent() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    boolean found = false;
    for (Student s : students) {
        if (s.regNo == regNo) {
            System.out.println("\nStudent Found!");
            s.display();
            found = true;
            break;
        }
    }
    if (!found) {
        System.out.println("Student Not Found!");
    }
}
```

`searchStudent()` reads a target registration number and performs a linear scan of the students list, comparing `s.regNo` to the input with `==`, which is a safe comparison here because `regNo` is a primitive `int` rather than a boxed object. As soon as a match is found, the method prints the record with `display()`, sets a found flag, and immediately breaks out of the loop to avoid scanning the remainder of the list unnecessarily. If the loop completes without a match, the found flag remains false and a "Student Not Found!" message is printed.

Edge cases / things to watch for:

- Search is $O(n)$ in the number of students because it is a straightforward linear scan; there is no index or map keyed by `regNo`.
- If duplicate registration numbers exist in the list (see `addStudent()` above), only the first one encountered is returned.

8.4 updateStudent()

```
static void updateStudent() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    for (Student s : students) {
        if (s.regNo == regNo) {
            System.out.print("Enter New Name: ");
            s.name = sc.next();
            System.out.print("Enter New Branch: ");
            s.branch = sc.next();
            System.out.print("Enter New Marks: ");
            s.marks = sc.nextInt();
            System.out.println("Student Updated Successfully!");
            return;
        }
    }
}
```

```

        }
    }
    System.out.println("Student Not Found!");
}

```

updateStudent() looks up a student by registration number using the same linear-scan pattern as searchStudent(). Once the matching Student object is found, its name, branch, and marks fields are overwritten directly (because Student's fields are package-private and this method lives in the same file/package) with newly entered values, and the method returns immediately with a success message. If no match is found, the loop finishes naturally and a "Student Not Found!" message is printed instead.

Edge cases / things to watch for:

- Only name, branch, and marks can be changed; regNo, totalClasses, and attendedClasses have no update path in the current menu, so correcting an attendance figure requires deleting and re-adding the record.
- The early return inside the loop means the method touches at most one record even if duplicate registration numbers exist.

8.5 deleteStudent()

```

static void deleteStudent() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    for (Student s : students) {
        if (s.regNo == regNo) {
            students.remove(s);
            System.out.println("Student Deleted Successfully!");
            return;
        }
    }
    System.out.println("Student Not Found!");
}

```

deleteStudent() again performs the same linear-scan lookup pattern, and on a match calls ArrayList.remove(Object), which removes the first element that is .equals() to the given reference. Since Student does not override equals(), this falls back to reference equality, which is safe here because s is the actual object drawn from the list during iteration.

Edge cases / things to watch for:

- Structurally modifying a list (removing an element) while iterating over it with a for-each loop can throw a ConcurrentModificationException in general; this code avoids that hazard only because it calls return immediately after the single remove() call, so the enhanced for-loop is never resumed after the structural change.
- As with search and update, only the first record matching the given regNo is affected.

8.6 calculateGrade()

```

static void calculateGrade() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    for (Student s : students) {
        if (s.regNo == regNo) {

```

```

        System.out.println("Marks: " + s.marks);
        System.out.println("Grade: " + s.getGrade());
        return;
    }
}
System.out.println("Student Not Found!");
}

```

calculateGrade() finds a student by registration number and prints their raw marks alongside the letter grade returned by Student.getGrade(). This duplicates part of what display() already shows, but is provided as a standalone menu option (option 6) for a user who only wants grade information without the full record printout.

8.7 calculatePercentage()

```

static void calculatePercentage() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    for (Student s : students) {
        if (s.regNo == regNo) {
            // Assuming marks are out of 100
            double percentage = s.marks;
            System.out.println("Percentage: " + percentage + "%");
            return;
        }
    }
    System.out.println("Student Not Found!");
}

```

calculatePercentage() is the simplest of the analytical methods: as the inline comment states, it assumes marks are already expressed out of 100, so the "percentage" it prints is numerically identical to the marks field, merely widened from int to double and suffixed with a percent sign.

Edge cases / things to watch for:

- This method does not compute a percentage from multiple subjects or a maximum-marks value; it is a direct pass-through of the marks field, so if marks were ever entered on a different scale (say, out of 50), the printed "percentage" would be misleading.

8.8 login()

```

static boolean login() {
    System.out.println("\n===== STUDENT LOGIN =====");
    System.out.print("Enter Username: ");
    String username = sc.next();
    System.out.print("Enter Password: ");
    String password = sc.next();
    if (username.equals("student")
        && password.equals("1234")) {
        System.out.println("Login Successful!");
        return true;
    } else {
        System.out.println("Invalid Username or Password!");
        return false;
    }
}

```

login() is called once, at the very start of main(), before the menu is ever shown. It reads a username and password as plain tokens and compares them against the hard-coded credentials "student" and

"1234" using `String.equals()`. If both match, it prints a success message and returns `true`, allowing `main()` to proceed into the menu loop; otherwise it prints a failure message and returns `false`, causing `main()` to return and the program to terminate without ever showing the menu.

Edge cases / things to watch for:

- The credentials are hard-coded directly in the source, which is appropriate only for a learning exercise; they are visible to anyone with access to the `.java` or compiled `.class` file and are not hashed or stored securely.
- The password is echoed as plain text in the terminal as it is typed, since `Scanner` has no built-in masked-input capability.
- There is only a single login attempt: entering the wrong credentials once ends the program rather than re-prompting.

8.9 attendance()

```
static void attendance() {
    System.out.print("Enter Registration Number: ");
    int regNo = sc.nextInt();
    for (Student s : students) {
        if (s.regNo == regNo) {
            double percentage = s.getAttendance();
            System.out.println("Attendance: " + percentage + "%");
            if (percentage >= 75)
                System.out.println("Status: Eligible");
            else
                System.out.println("Status: Not Eligible");
            return;
        }
    }
    System.out.println("Student Not Found!");
}
```

`attendance()` looks up a student, calls `Student.getAttendance()` to obtain the attendance percentage, prints it, and then applies a fixed 75% threshold: at or above 75% the student is reported as "Eligible" (commonly meaning eligible to sit an examination in many academic institutions), and below 75% as "Not Eligible". This mirrors `calculateGrade()` in that it exposes, as a dedicated menu option, a single derived value that is also visible inside the fuller `display()` printout.

8.10 main(String[] args)

```
public static void main(String[] args) {
    if (!login()) {
        return;
    }
    int choice;
    do {
        System.out.println("\n=====");
        System.out.println("      STUDENT MANAGEMENT SYSTEM");
        System.out.println("=====");
        System.out.println("1. Add Student");
        System.out.println("2. Display Students");
        System.out.println("3. Search Student");
        System.out.println("4. Update Student");
        System.out.println("5. Delete Student");
        System.out.println("6. Calculate Grade");
        System.out.println("7. Calculate Percentage");
        System.out.println("8. Attendance");
```



```

        System.out.println("9. Exit");
        System.out.println("=====");
        System.out.print("Enter your choice: ");
        choice = sc.nextInt();
        switch (choice) {
            case 1: addStudent(); break;
            case 2: displayStudents(); break;
            case 3: searchStudent(); break;
            case 4: updateStudent(); break;
            case 5: deleteStudent(); break;
            case 6: calculateGrade(); break;
            case 7: calculatePercentage(); break;
            case 8: attendance(); break;
            case 9: System.out.println("Thank You!"); break;
            default: System.out.println("Invalid Choice!");
        }
    } while (choice != 9);
}

```

main() is the program's entry point and orchestrator. It first calls login(); if that returns false the method returns immediately and the program ends without ever showing a menu. If login succeeds, it enters a do-while loop that is guaranteed to execute at least once: on each iteration it prints the menu banner and nine numbered options, reads an integer choice, and dispatches to the corresponding method through a switch statement. Every case except the default falls through to a break, and choice 9 both prints a farewell message and, via the loop's own while(choice != 9) condition, causes the loop to terminate after that iteration completes.

Any value that does not match cases 1 through 9 falls into the default branch, which prints "Invalid Choice!" and simply lets the loop repeat, re-displaying the menu.

9. Menu Flow & Control Logic

The overall control flow of the program can be summarised as a two-stage process: an authentication gate, followed by an unbounded menu loop that keeps running until the user explicitly chooses to exit. The textual flow diagram below traces this from program start to program end.

```
START
|
v
Call login()
|
+-- invalid credentials --> print "Invalid Username or Password!" --> END
|
v (valid credentials)
print "Login Successful!"
|
v
+-----+
| REPEAT (do-while, condition: choice != 9) |
|   print menu banner (options 1-9)         |
|   read choice = sc.nextInt()               |
|   switch(choice):                          |
|     1 -> addStudent()                      |
|     2 -> displayStudents()                 |
|     3 -> searchStudent()                   |
|     4 -> updateStudent()                   |
|     5 -> deleteStudent()                   |
|     6 -> calculateGrade()                  |
|     7 -> calculatePercentage()              |
|     8 -> attendance()                      |
|     9 -> print "Thank You!"                 |
|     default -> print "Invalid Choice!"      |
+-----+
|
v (loop exits when choice == 9)
END
```

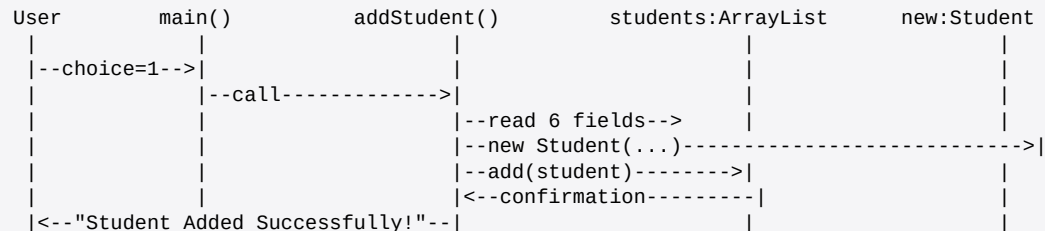
Because the loop condition is checked only after the body executes (do-while), the menu is always shown at least once after a successful login, even before the user has made any choice at all. Every menu operation is a self-contained method call: control always returns to the top of the loop after each operation completes, there is no nested menu or sub-state to track, and the only piece of state that persists across iterations is the shared students list.

One consequence of this simple structure is that an invalid (non-integer) response to "Enter your choice:" will cause `Scanner.nextInt()` to throw an unhandled `InputMismatchException`, since the program does not wrap this call in a try/catch block; this will terminate the program abruptly rather than showing the "Invalid Choice!" message, which only triggers for a syntactically valid integer that simply does not match any case.

9.1 Sequence Diagrams for Key Operations

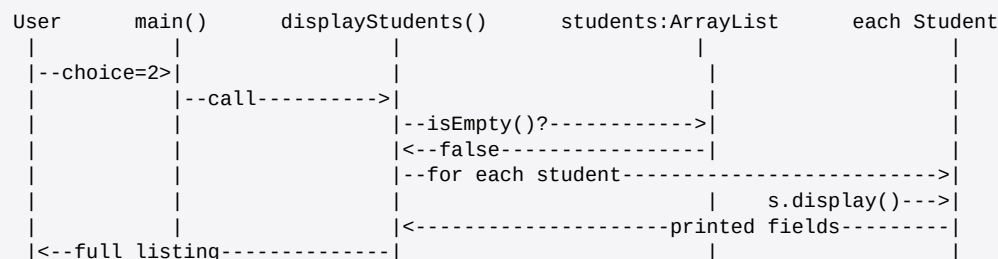
To make the interaction between the user, main(), the individual operation methods, the shared students list, and the Student objects themselves fully explicit, this subsection presents a simple text-based sequence diagram for each of the nine menu operations, followed by a short narrative walk-through of the objects and messages involved.

Sequence: Add Student (Option 1)



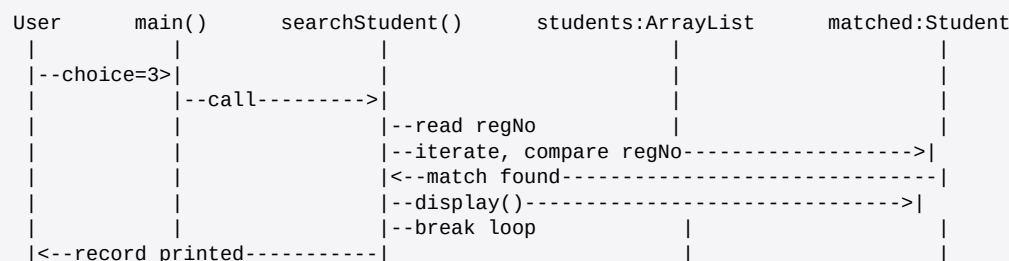
The user's choice of 1 routes control from main() into addStudent(). That method owns the entire interaction with the user for this operation: it reads six values directly from the shared Scanner, constructs exactly one new Student object with those values, and appends it to the shared students list with a single ArrayList.add() call before printing a confirmation. No other method or object is involved.

Sequence: Display Students (Option 2)



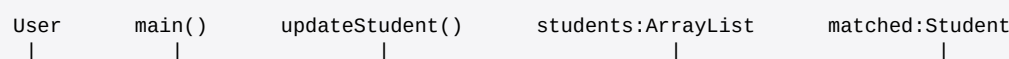
displayStudents() first asks the list whether it is empty; if not, it iterates over every stored Student in insertion order and delegates the actual field-by-field printing to that object's own display() method, which in turn calls back into the same Student instance for getGrade() and getAttendance() before returning control to the loop.

Sequence: Search Student (Option 3)



searchStudent() reads the target registration number once, then scans the list element by element. As soon as an element's regNo field equals the target, that Student's display() method is invoked and the loop is broken; the remaining elements of the list, if any, are never inspected.

Sequence: Update Student (Option 4)



`updateStudent()` finds the target `Student` object by reference during the same scan used for searching, then mutates that object's `name`, `branch`, and `marks` fields in place. Because Java objects are passed and held by reference inside the `ArrayList`, this in-place mutation is immediately visible to every other method that later reads the same list, without needing to explicitly write the object back into the list.

User	main()	deleteStudent()	students:ArrayList	matched:Student
	--choice=5>			
		--call----->		
		--read regNo		
		--iterate, compare regNo----->		
		<--match found----->		
		--remove(s)----->		
		--return		
	<--"Student Deleted..."----->			

User	main()	calculateGrade()	students:ArrayList	matched:Student
--choice=6>				
	--call----->			
		--read regNo		
		--iterate, compare regNo----->		
		<-match found----->		
		--getGrade()----->		
		<-char grade----->		
<--marks & grade printed----				

User	main()	calculatePercentage()	students:ArrayList	matched:Student
--choice=7>				
	--call----->			
		--read regNo		
		--iterate, compare regNo----->		
		<-match found----->		
		--read s.marks directly----->		
<--percentage printed----->				

Sequence: Login (Program Start-up)

User	main()	login()
	--call----->	
		--print login banner
--username---->		
	----->	--read username
--password---->		
	----->	--read password
	----->	--compare to "student"/"1234"
<--result msg--	<--true/false-----	

login() is the very first thing main() calls. It owns its own small, self-contained input/compare/report cycle and returns a single boolean; main() uses that boolean to decide whether to proceed into the menu loop or terminate immediately.

Sequence: Attendance (Option 8)

User	main()	attendance()	students:ArrayList	matched:Student
--choice=8>				
	--call----->			
		--read regNo		
		--iterate, compare regNo----->		
		<--match found-----		
		--getAttendance()----->		
		<--double percentage-----		
		--compare percentage >= 75		
<--attendance & status-----				

attendance() delegates the percentage calculation to `Student.getAttendance()` and then applies its own 75% threshold comparison locally to decide which of the two status strings to print, keeping the eligibility policy inside the controller rather than inside the `Student` class itself.

9.2 Glossary of Terms

The short glossary below defines Java and object-oriented programming terms used throughout this document, for readers who may be encountering some of them for the first time.

Class	A blueprint for creating objects that bundles data (fields) and behaviour (methods) together; <code>Student</code> and <code>StudentManagementSystem</code> are the two classes in this program.
Object	A concrete instance of a class created at runtime; each call to <code>"new Student(...)"</code> produces one object representing a single student.
Field	A variable declared inside a class that holds part of an object's state, such as <code>regNo</code> or <code>marks</code> .
Method	A named block of code declared inside a class that can be invoked to perform an action or compute a value, such as <code>getGrade()</code> or <code>addStudent()</code> .
Constructor	A special method with the same name as its class, invoked with <code>"new"</code> , that initialises a newly created object's fields.
Static member	A field or method that belongs to the class itself rather than to any particular object; <code>students</code> , <code>sc</code> , and every method in <code>StudentManagementSystem</code> are static.
ArrayList	A resizable, ordered collection provided by <code>java.util</code> , used here to store an arbitrary number of <code>Student</code> objects without a fixed capacity.
Scanner	A utility class from <code>java.util</code> used to read formatted input, in this program from the console (<code>System.in</code>).
Enhanced for-loop (for-each)	A compact loop syntax, <code>"for (Student s : students)"</code> , that iterates over every element of a collection in order without needing an explicit index variable.
switch statement	A control-flow construct that selects one of several code blocks (cases) to execute based on the value of an expression, used in <code>main()</code> to dispatch on the user's menu choice.
do-while loop	A loop whose body executes at least once before its continuation condition is first checked, used to guarantee the menu is shown after a successful login.
Encapsulation	The object-oriented principle of bundling data with the methods that operate on it, illustrated here by <code>Student</code> owning both its raw fields and the <code>getGrade()/getAttendance()</code> logic that derives meaning from them.

Linear search	A search technique that inspects each element of a collection in order until a match is found or the collection is exhausted; used by <code>searchStudent()</code> , <code>updateStudent()</code> , <code>deleteStudent()</code> , <code>calculateGrade()</code> , <code>calculatePercentage()</code> , and <code>attendance()</code> .
InputMismatchException	A runtime exception thrown by <code>Scanner</code> when the next token in the input does not match the type requested (for example, calling <code>nextInt()</code> when the user typed letters).
NaN / Infinity	Special floating-point values defined by the IEEE 754 standard that Java produces instead of throwing an exception when a double division has no ordinary numeric result, such as <code>0.0/0.0</code> (NaN) or a non-zero value divided by <code>0.0</code> (Infinity).

10. Sample Execution Transcripts

This section shows realistic, hand-traced console transcripts for a typical session, covering login and each of the nine menu options in turn. User-entered values are shown after the prompts exactly as they would appear on screen (Scanner echoes nothing extra; the terminal itself displays what is typed).

10.1 Login

```
===== STUDENT LOGIN =====  
Enter Username: student  
Enter Password: 1234  
Login Successful!
```

10.2 Option 1 — Add Student

```
Enter your choice: 1  
Enter Registration Number: 101  
Enter Student Name: Ananya  
Enter Branch: CSE  
Enter Marks: 88  
Enter Total Classes: 60  
Enter Attended Classes: 54  
Student Added Successfully!
```

10.3 Option 2 — Display Students

```
Enter your choice: 2  
  
===== STUDENT DETAILS =====  
Reg No      : 101  
Name        : Ananya  
Branch      : CSE  
Marks       : 88  
Grade       : B  
Attendance  : 90.0%  
-----
```

10.4 Option 3 — Search Student

```
Enter your choice: 3  
Enter Registration Number: 101  
  
Student Found!  
Reg No      : 101  
Name        : Ananya  
Branch      : CSE  
Marks       : 88  
Grade       : B  
Attendance  : 90.0%  
-----
```

10.5 Option 4 — Update Student

```
Enter your choice: 4  
Enter Registration Number: 101  
Enter New Name: Ananya  
Enter New Branch: CSE  
Enter New Marks: 93  
Student Updated Successfully!
```

10.6 Option 6 — Calculate Grade (after the update above)

```
Enter your choice: 6  
Enter Registration Number: 101  
Marks: 93  
Grade: A
```


10.7 Option 7 — Calculate Percentage

```
Enter your choice: 7
Enter Registration Number: 101
Percentage: 93.0%
```

10.8 Option 8 — Attendance

```
Enter your choice: 8
Enter Registration Number: 101
Attendance: 90.0%
Status: Eligible
```

10.9 Option 5 — Delete Student

```
Enter your choice: 5
Enter Registration Number: 101
Student Deleted Successfully!
```

10.10 Option 2 — Display Students (after deletion)

```
Enter your choice: 2
No Students Available!
```

10.11 Option 9 — Exit

```
Enter your choice: 9

=====
      STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Calculate Grade
7. Calculate Percentage
8. Attendance
9. Exit
=====
Enter your choice: 9
Thank You!
```

Note that the menu banner is printed again immediately before the choice that ends the loop, because the banner is printed at the top of every iteration, including the final one where the user types 9.

10.12 Unsuccessful Login Attempt

```
===== STUDENT LOGIN =====
Enter Username: admin
Enter Password: admin
Invalid Username or Password!
```

After an unsuccessful login, `main()` returns immediately: the menu is never displayed and the program exits, so the only way back in is to restart the program and try the credentials again.

10.13 Searching for a Non-Existent Registration Number

```
Enter your choice: 3
Enter Registration Number: 999
Student Not Found!
```

10.14 Attendance Below the Eligibility Threshold

```
Enter your choice: 1
Enter Registration Number: 102
Enter Student Name: Rohit
Enter Branch: ECE
Enter Marks: 62
Enter Total Classes: 60
Enter Attended Classes: 40
Student Added Successfully!
```

```
Enter your choice: 8
Enter Registration Number: 102
Attendance: 66.66666666666666%
Status: Not Eligible
```

This transcript illustrates two behaviours worth noting together: first, Java's default double-to-String conversion prints many decimal digits rather than rounding to two decimal places, since the program does not use any formatting utility such as `String.format` or `DecimalFormat`; second, at 66.67% attendance the student falls below the 75% cut-off and is correctly reported as "Not Eligible".

11. Test Cases & Validation

The table below lists functional test cases that exercise every menu option, along with a smaller table of boundary and negative test cases that probe how the program behaves outside of its documented happy path.

11.1 Functional Test Cases

ID	Description	Input	Expected Result	Status
TC-01	Login with correct credentials	username=student, password=12345	"Login Successful!" and menu is shown	Pass
TC-02	Login with incorrect credentials	username=admin, password=admin	"Invalid Username or Password!" and program ends	Pass
TC-03	Add a new student	regNo=101, valid fields	"Student Added Successfully!"	Pass
TC-04	Display students when list has records	choice=2 after TC-03	Full record printed with derived grade & attendance	Pass
TC-05	Display students when list is empty	choice=2 on a fresh run	"No Students Available!"	Pass
TC-06	Search for an existing student	regNo=101	Matching record is printed	Pass
TC-07	Search for a non-existent student	regNo=999	"Student Not Found!"	Pass
TC-08	Update an existing student	regNo=101, new name/branch/marks	"Student Updated Successfully!" and later records reflect new values	Pass
TC-09	Update a non-existent student	regNo=999	"Student Not Found!"	Pass
TC-10	Delete an existing student	regNo=101	"Student Deleted Successfully!"	Pass
TC-11	Delete a non-existent student	regNo=999	"Student Not Found!"	Pass
TC-12	Calculate grade for marks=93	regNo of that student	"Grade: A"	Pass
TC-13	Calculate grade for marks=62	regNo of that student	"Grade: D"	Pass
TC-14	Calculate percentage	regNo with marks=93	"Percentage: 93.0%"	Pass
TC-15	Attendance at or above 75%	attended=54, total=60	"Status: Eligible"	Pass
TC-16	Attendance below 75%	attended=40, total=60	"Status: Not Eligible"	Pass
TC-17	Exit the program	choice=9	"Thank You!" and loop terminates	Pass

11.2 Boundary & Negative Test Cases

ID	Description	Input	Observed Behaviour	Status
TC-18	Menu choice out of range	choice=15	"Invalid Choice!" and menu re-displays	Pass
TC-19	Non-numeric menu choice	choice=abc	InputMismatchException, program crashes	Fail (unhandled)
TC-20	Marks exactly on a grade boundary	marks=90	"Grade: A" (boundary is inclusive)	Pass
TC-21	Marks exactly on a grade boundary	marks=89	"Grade: B"	Pass
TC-22	Attendance exactly at threshold	attended=45, total=60 (75%)	"Status: Eligible" (boundary is inclusive, >=)	Pass
TC-23	Total classes is zero	total=0, attended=0	Attendance prints as NaN%, no error thrown	Fail (data quality)
TC-24	Negative marks entered	marks=-10	"Grade: F" is returned, no rejection	Fail (no validation)

ID	Description	Input	Observed Behaviour	Status
TC-25	Attended greater than total classes	attended=70, total=60	Attendance prints as 116.67%, no error thrown	Fail (no validation)
TC-26	Duplicate registration number added twice	regNo=101	Both records are stored; search/update/delete fails (read/write check)	Fail (read/write check)
TC-27	Name containing a space	name="Mary Ann"	Only "Mary" is stored as the name	Fail (tokenised input)

The "Fail" entries above are not implementation bugs in the sense of code that does not compile or crashes unpredictably; rather, they document places where the program does exactly what its code says but does not defend against invalid or unexpected input. These are catalogued in detail in Section 13 (Known Limitations) and addressed with concrete suggestions in Section 14 (Suggested Enhancements).

12. Grading & Attendance Policy Reference

12.1 Grade Boundaries

Marks Range	Grade	Condition in Code
90 – 100	A	marks >= 90
80 – 89	B	marks >= 80 (and < 90)
70 – 79	C	marks >= 70 (and < 80)
60 – 69	D	marks >= 60 (and < 70)
Below 60 (including negative values)	F	falls through every else-if

12.2 Attendance Eligibility

Attendance Percentage	Status	Condition in Code
75% or above	Eligible	percentage >= 75
Below 75%	Not Eligible	percentage < 75

Both thresholds (60/70/80/90 for grading, 75 for attendance eligibility) are hard-coded as literal numbers directly inside the if/else-if chains rather than being declared as named constants. This is acceptable for a small teaching project but means that changing a policy value, such as the eligibility cut-off, requires editing the source code and recompiling rather than adjusting a configuration value.

12.3 Complexity & Performance Analysis

Although this is a small teaching program unlikely to be run against a large dataset, it is still useful to reason about the algorithmic cost of each operation, since the same lookup pattern (a linear scan of the students list) is reused across most of the menu's functionality.

Operation	Time Complexity	Space Complexity	Notes
addStudent()	$O(1)$ amortised	$O(1)$ additional	<code>ArrayList.add()</code> is amortised constant time; occasional internal array re...
displayStudents()	$O(n)$	$O(1)$ additional	Must visit every stored student exactly once to print it.
searchStudent()	$O(n)$ worst case	$O(1)$ additional	Linear scan; best case $O(1)$ if the match is the first element.
updateStudent()	$O(n)$ worst case	$O(1)$ additional	Same linear scan as search, plus constant-time field mutation once fou...
deleteStudent()	$O(n)$	$O(1)$ additional	Linear scan to find the element, plus $O(n)$ internal shift performed by A...
calculateGrade()	$O(n)$ worst case	$O(1)$ additional	Linear scan, then $O(1)$ grade computation.
calculatePercentage()	$O(n)$ worst case	$O(1)$ additional	Linear scan, then $O(1)$ field read.
attendance()	$O(n)$ worst case	$O(1)$ additional	Linear scan, then $O(1)$ percentage computation.
login()	$O(1)$	$O(1)$	Two fixed string comparisons regardless of any other state.

In every case, n refers to the number of student records currently stored in the students list. For the class sizes this program is realistically used with — tens to a few hundred students — a linear scan is imperceptibly fast; the $O(n)$ cost only becomes a practical concern if the system were scaled to many thousands of records, at which point switching the backing structure to a `HashMap` keyed by registration number, as suggested in Section 14.2, would bring search, update, and delete down to average $O(1)$.

13. Known Limitations

No attempt is made in this report to hide the rough edges of a compact console project. The following is a candid list of limitations observed while documenting the code, grouped by theme.

13.1 Data Persistence

All student data is kept in a single `ArrayList` that exists only for the lifetime of the running Java process. As soon as the program exits — whether via option 9, a failed login, or a crash — every record added during that session is lost. There is no file, database, or serialization mechanism of any kind.

13.2 Input Validation

- Marks are not checked to fall within a sensible 0–100 range.
- `attendedClasses` is not checked against `totalClasses`, so attendance figures above 100% are possible.
- `totalClasses` is not checked to be non-zero, which allows a division-by-zero condition inside `getAttendance()` that surfaces as `NaN` or `Infinity` in the console output rather than a clear error message.
- Registration numbers are not checked for uniqueness, so duplicate records can exist side by side.
- Names and branch names are read with `Scanner.next()`, which stops at the first whitespace character, so multi-word names or branches are silently truncated.

13.3 Error Handling

- None of the `Scanner.nextInt()` calls are wrapped in a try/catch block, so typing a non-numeric value at any numeric prompt (including the main menu choice) throws an unhandled `InputMismatchException` and terminates the program abruptly.
- There is no mechanism to recover from a bad input and re-prompt the user within the same field; the only recovery path is restarting the whole program.

13.4 Security

- The login credentials are hard-coded directly in the source (username "student", password "1234"), visible to anyone who can read the `.java` file or decompile the `.class` file.
- Passwords are compared and stored as plain `String` objects with no hashing, salting, or masking of console input while typing.
- There is only one account; the system cannot distinguish between different users or grant different permission levels.

13.5 Usability & Presentation

- Floating-point results (attendance percentage, percentage figure) are printed with Java's default double formatting, which can show many trailing decimal digits (e.g. 66.66666666666666%) instead of a clean, rounded figure.

- There is no way to list students sorted by name, branch, or marks; the display order always matches insertion order.
- The menu, once shown, cannot be cleared from the screen, so a long session produces a continuously scrolling console log rather than a refreshed view.

14. Suggested Enhancements

The limitations catalogued above map directly onto a roadmap of enhancements that would take this project from a teaching exercise toward something closer to production-ready software. They are grouped below by the effort they would likely require.

14.1 Low-Effort Improvements

- Wrap every `Scanner.nextInt()` call in a try/catch for `InputMismatchException`, print a friendly error message, and re-prompt instead of letting the exception propagate and crash the program.
- Use `sc.nextLine()` (after consuming the trailing newline) instead of `sc.next()` for name and branch so that multi-word values are captured correctly.
- Format floating-point output with `String.format("%.2f", value)` so attendance and percentage figures always show exactly two decimal places.
- Add simple range checks on marks (0–100) and on `attendedClasses` versus `totalClasses`, rejecting or re-prompting for out-of-range values.
- Extract the magic numbers used for grade and eligibility thresholds into named constants near the top of the class, making future policy changes a one-line edit.

14.2 Medium-Effort Improvements

- Enforce unique registration numbers when adding a new student, either by scanning the existing list before insertion or by switching the storage structure to a `Map` keyed on `regNo`.
- Add a re-prompt loop to `login()` so an incorrect username/password does not immediately end the program, perhaps allowing a fixed number of attempts before locking out.
- Add sorting options to `displayStudents()` (by name, by marks, by attendance) using `Collections.sort` with a custom `Comparator`.
- Introduce a simple text-file persistence layer using standard I/O (e.g. writing/reading a CSV file) so that data survives between runs without requiring a full database.

14.3 Larger Architectural Improvements

- Replace the console interface with a graphical user interface built in JavaFX or Swing, keeping the `Student` class and its business logic largely unchanged.
- Introduce a real relational database (e.g. MySQL, PostgreSQL, or SQLite) accessed through JDBC, replacing the in-memory `ArrayList` with persistent tables.
- Support multiple user roles (administrator versus student) with different permitted operations, backed by a proper authentication mechanism rather than a single hard-coded credential pair.
- Expose the same functionality as a REST API using a framework such as Spring Boot, enabling a web or mobile client to be built on top of the same business logic.
- Add automated unit tests (e.g. with JUnit) for `Student.getGrade()`, `Student.getAttendance()`, and the controller methods, so future changes can be verified without manual console testing.

15. Advantages of the System

Despite the limitations discussed above, the program succeeds well at what it sets out to be: a compact, understandable demonstration of core Java and object-oriented programming ideas.

- **Simplicity:** the entire program fits in two classes and under 300 lines, making it easy to read from top to bottom in a single sitting.
- **No external dependencies:** it compiles and runs with nothing more than a standard JDK, with no build tool, framework, or library to install.
- **Clear separation of concerns:** data (Student) and orchestration/behaviour (StudentManagementSystem) are kept in separate classes, illustrating basic object-oriented design.
- **Immediate feedback:** every operation prints a clear confirmation or error message, so a user always knows whether their action succeeded.
- **Derived values stay consistent:** because grade and attendance are computed on demand rather than cached, they can never go stale relative to the underlying marks or attendance figures.
- **Good teaching value:** the codebase is an approachable case study for discussing ArrayList usage, Scanner-based I/O, switch statements, do-while loops, and simple authentication logic.

16. Conclusion

The Student Management System documented in this report is a compact, single-file Java console application that demonstrates the essentials of object-oriented design: a data-holding `Student` class, a controller class that manages a collection of students, and a menu-driven loop that exposes create, read, update, and delete operations alongside two derived analytical features (grading and attendance eligibility) gated behind a simple login check.

This documentation has walked through the system's requirements, its two-class architecture, every individual method with its logic and edge cases, realistic sample transcripts covering all nine menu options plus login success and failure, a structured set of functional and boundary test cases, and an honest accounting of the program's current limitations alongside concrete, prioritised suggestions for improving it. Taken together, these sections should give a reader — whether an evaluator, a fellow student, or the original author revisiting the project later — everything needed to understand, operate, test, and extend the system with confidence.

As a learning exercise, the project meets its goals well: it is small enough to fully understand, yet rich enough to demonstrate collections, control flow, encapsulation, and derived computed properties in a single, coherent program. The path from here to a more robust, production-style system is well-defined and incremental, as laid out in Section 14, making this an excellent foundation for further coursework or independent study.

17. Viva-Voce Question Bank

The questions below are representative of what an evaluator might ask about this project during a code review or viva-voce examination, together with concise model answers grounded in the actual source code.

Q1. Why is the `Student` class not declared public?

Only one public top-level class is allowed per `.java` file, and that role is taken by `StudentManagementSystem` because it contains the main method required to run the program. `Student` is therefore left with default (package-private) visibility, which is sufficient since it is used only within the same source file.

Q2. Why does `getAttendance()` cast `attendedClasses` to `(double)`?

Without the cast, `attendedClasses / totalClasses` would perform integer division in Java, truncating any fractional part and typically yielding 0 or 1 rather than a meaningful percentage. Casting the numerator to `double` forces Java to use floating-point division, preserving the fractional result before it is multiplied by 100.

Q3. What data structure holds the students, and why was it chosen over an array?

Students are held in an `ArrayList`. Unlike a fixed-size array, an `ArrayList` can grow dynamically as new students are added and shrink cleanly when a student is deleted, without the programmer needing to manage capacity or manually shift elements after a removal.

Q4. How does the program decide a student's grade?

`Student.getGrade()` applies a top-down if/else-if ladder against the marks field: 90 and above is 'A', 80–89 is 'B', 70–79 is 'C', 60–69 is 'D', and anything below 60 falls through to the final else and returns 'F'.

Q5. What happens if two students are added with the same registration number?

The program does not check for uniqueness, so both records are stored side by side in the `ArrayList`. Because search, update, and delete all use a for-each loop with an early break/return on the first match, only the first of the two duplicate records is ever reachable through those operations.

Q6. Why is only one `Scanner` object created for the whole program?

The `Scanner sc` field is declared once, statically, and reused by every method that needs console input. Opening multiple `Scanner` instances over the same `System.in` stream is a common source of bugs in beginner Java programs, since buffered input can be consumed unpredictably across instances; sharing one instance avoids that entirely.

Q7. What is the purpose of the do-while loop in `main()`, and why not a plain while loop?

A do-while loop guarantees the loop body — printing the menu and reading a choice — runs at least once. Since the menu must always be shown at least once immediately after a successful login (there is no prior value for choice to test), do-while is the natural fit; a plain while loop would need an artificial initial value for choice before the first check.

Q8. How is program exit implemented?

Choosing option 9 prints "Thank You!" inside the switch statement, and separately, the loop's own condition while (choice != 9) evaluates to false once choice equals 9, so the loop terminates naturally after that iteration completes; there is no explicit break or System.exit() call.

Q9. What would happen if a user typed a letter instead of a number at the main menu prompt?

sc.nextInt() would throw an unhandled InputMismatchException, since there is no surrounding try/catch. Because the exception is not caught anywhere in main(), it would propagate out of main and terminate the program with a stack trace rather than showing the "Invalid Choice!" message, which only applies to valid integers outside the 1–9 range.

Q10. How is the login implemented, and what are its weaknesses?

login() reads a username and password with Scanner.next() and compares them with String.equals() against the hard-coded values "student" and "1234". Its main weaknesses are that the credentials are visible in the source code, there is no hashing or masking, and a single failed attempt ends the program instead of allowing a retry.

Q11. Why does calculatePercentage() simply return the marks value?

The inline comment in the source states the assumption explicitly: marks are already assumed to be out of 100, so the raw marks value is numerically identical to a percentage on that same scale. The method widens it to a double and appends a percent sign for display, without any further calculation.

Q12. What is the eligibility threshold for attendance, and where is it defined?

The threshold is 75%, hard-coded as a literal inside an if statement in the attendance() method (if (percentage >= 75)). It is not stored as a named constant, so changing the policy requires editing this literal directly in the source.

Q13. Why might display() show 'Infinity' or 'NaN' for attendance in some cases?

If a student's totalClasses value is zero, getAttendance() divides by zero using floating-point arithmetic. Unlike integer division, floating-point division by zero in Java does not throw an exception; instead it yields Infinity (for a non-zero numerator) or NaN (for zero divided by zero), and this value is printed as-is.

Q14. Could this program be converted to use a database? What would change?

Yes. The Student class could remain almost unchanged as a plain data object, while the students ArrayList and the direct field manipulation in methods like addStudent(), updateStudent(), and deleteStudent() would be replaced with SQL statements executed through JDBC (or an ORM), turning the in-memory list into a persistent table.

Q15. What design pattern, if any, does this project loosely follow?

It loosely resembles a simplified Model–Controller split: Student acts as the model holding data and simple derived calculations, while StudentManagementSystem acts as both controller (menu dispatch and business logic) and view (all console output), since no separate view layer exists.

Q16. Why does updateStudent() not allow changing the registration number?

The registration number is treated as the stable key used to look up a record in the first place; allowing it to change during the same operation that uses it as a lookup key would complicate the logic, so the source only exposes name, branch, and marks as editable fields.

18. Appendix A — Complete Source Code

The complete, unmodified source code of StudentManagementSystem.java is reproduced below for reference, exactly as it was provided for this documentation.

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  class Student {
5      int regNo;
6      String name;
7      String branch;
8      int marks;
9      int totalClasses;
10     int attendedClasses;
11
12     Student(int regNo, String name, String branch, int marks,
13             int totalClasses, int attendedClasses) {
14         this.regNo = regNo;
15         this.name = name;
16         this.branch = branch;
17         this.marks = marks;
18         this.totalClasses = totalClasses;
19         this.attendedClasses = attendedClasses;
20     }
21
22     double getAttendance() {
23         return ((double) attendedClasses / totalClasses) * 100;
24     }
25
26     char getGrade() {
27         if (marks >= 90)
28             return 'A';
29         else if (marks >= 80)
30             return 'B';
```

```

31         else if (marks >= 70)
32             return 'C';
33         else if (marks >= 60)
34             return 'D';
35         else
36             return 'F';
37     }
38
39     void display() {
40         System.out.println("Reg No      : " + regNo);
41         System.out.println("Name       : " + name);
42         System.out.println("Branch     : " + branch);
43         System.out.println("Marks      : " + marks);
44         System.out.println("Grade      : " + getGrade());
45         System.out.println("Attendance : " + getAttendance() + "%");
46         System.out.println("-----");
47     }
48 }
49
50 public class StudentManagementSystem {
51
52     static ArrayList<Student> students = new ArrayList<>();
53     static Scanner sc = new Scanner(System.in);
54
55     // 1. Add Student
56     static void addStudent() {
57
58         System.out.print("Enter Registration Number: ");
59         int regNo = sc.nextInt();
60

```



```

61     System.out.print("Enter Student Name: ");
62     String name = sc.next();
63
64     System.out.print("Enter Branch: ");
65     String branch = sc.next();
66
67     System.out.print("Enter Marks: ");
68     int marks = sc.nextInt();
69
70     System.out.print("Enter Total Classes: ");
71     int total = sc.nextInt();
72
73     System.out.print("Enter Attended Classes: ");
74     int attended = sc.nextInt();
75
76     students.add(new Student(
77         regNo, name, branch, marks, total, attended));
78
79     System.out.println("Student Added Successfully!");
80 }
81
82 // 2. Display Students
83 static void displayStudents() {
84
85     if (students.isEmpty()) {
86         System.out.println("No Students Available!");
87         return;
88     }
89
90     System.out.println("\n===== STUDENT DETAILS =====");

```

```

91
92     for (Student s : students) {
93         s.display();
94     }
95 }
96
97 // 3. Search Student
98 static void searchStudent() {
99
100     System.out.print("Enter Registration Number: ");
101     int regNo = sc.nextInt();
102
103     boolean found = false;
104
105     for (Student s : students) {
106
107         if (s.regNo == regNo) {
108             System.out.println("\nStudent Found!");
109             s.display();
110             found = true;
111             break;
112         }
113     }
114
115     if (!found) {
116         System.out.println("Student Not Found!");
117     }
118 }
119
120 // 4. Update Student

```

```

121     static void updateStudent() {
122
123         System.out.print("Enter Registration Number: ");
124         int regNo = sc.nextInt();
125
126         for (Student s : students) {
127
128             if (s.regNo == regNo) {
129
130                 System.out.print("Enter New Name: ");
131                 s.name = sc.next();
132
133                 System.out.print("Enter New Branch: ");
134                 s.branch = sc.next();
135
136                 System.out.print("Enter New Marks: ");
137                 s.marks = sc.nextInt();
138
139                 System.out.println("Student Updated Successfully!");
140                 return;
141             }
142         }
143
144         System.out.println("Student Not Found!");
145     }
146
147     // 5. Delete Student
148     static void deleteStudent() {
149
150         System.out.print("Enter Registration Number: ");

```

```

151         int regNo = sc.nextInt();
152
153         for (Student s : students) {
154
155             if (s.regNo == regNo) {
156                 students.remove(s);
157                 System.out.println("Student Deleted Successfully!");
158                 return;
159             }
160         }
161
162         System.out.println("Student Not Found!");
163     }
164
165     // 6. Grade
166     static void calculateGrade() {
167
168         System.out.print("Enter Registration Number: ");
169         int regNo = sc.nextInt();
170
171         for (Student s : students) {
172
173             if (s.regNo == regNo) {
174                 System.out.println("Marks: " + s.marks);
175                 System.out.println("Grade: " + s.getGrade());
176                 return;
177             }
178         }
179
180         System.out.println("Student Not Found!");

```

```

181     }
182
183     // 7. Percentage
184     static void calculatePercentage() {
185
186         System.out.print("Enter Registration Number: ");
187         int regNo = sc.nextInt();
188
189         for (Student s : students) {
190
191             if (s.regNo == regNo) {
192
193                 // Assuming marks are out of 100
194                 double percentage = s.marks;
195
196                 System.out.println("Percentage: "
197                     + percentage + "%");
198
199                 return;
200             }
201         }
202
203         System.out.println("Student Not Found!");
204     }
205
206     // 8. Login
207     static boolean login() {
208
209         System.out.println("\n===== STUDENT LOGIN =====");
210

```

```

211     System.out.print("Enter Username: ");
212     String username = sc.next();
213
214     System.out.print("Enter Password: ");
215     String password = sc.next();
216
217     if (username.equals("student")
218         && password.equals("1234")) {
219
220         System.out.println("Login Successful!");
221         return true;
222     }
223     } else {
224
225         System.out.println("Invalid Username or Password!");
226         return false;
227     }
228 }
229
230 // 9. Attendance
231 static void attendance() {
232
233     System.out.print("Enter Registration Number: ");
234     int regNo = sc.nextInt();
235
236     for (Student s : students) {
237
238         if (s.regNo == regNo) {
239
240             double percentage = s.getAttendance();

```

```

241
242         System.out.println("Attendance: "
243             + percentage + "%");
244
245         if (percentage >= 75)
246             System.out.println("Status: Eligible");
247         else
248             System.out.println("Status: Not Eligible");
249
250         return;
251     }
252 }
253
254     System.out.println("Student Not Found!");
255 }
256
257 // 10. Main Menu
258 public static void main(String[] args) {
259
260     if (!login()) {
261         return;
262     }
263
264     int choice;
265
266     do {
267
268         System.out.println("\n=====");
269         System.out.println("      STUDENT MANAGEMENT SYSTEM");
270         System.out.println("=====");

```

```

271         System.out.println("1. Add Student");
272         System.out.println("2. Display Students");
273         System.out.println("3. Search Student");
274         System.out.println("4. Update Student");
275         System.out.println("5. Delete Student");
276         System.out.println("6. Calculate Grade");
277         System.out.println("7. Calculate Percentage");
278         System.out.println("8. Attendance");
279         System.out.println("9. Exit");
280         System.out.println("=====");
281
282         System.out.print("Enter your choice: ");
283         choice = sc.nextInt();
284
285         switch (choice) {
286
287             case 1:
288                 addStudent();
289                 break;
290
291             case 2:
292                 displayStudents();
293                 break;
294
295             case 3:
296                 searchStudent();
297                 break;
298
299             case 4:
300                 updateStudent();

```



```

301             break;
302
303         case 5:
304             deleteStudent();
305             break;
306
307         case 6:
308             calculateGrade();
309             break;
310
311         case 7:
312             calculatePercentage();
313             break;
314
315         case 8:
316             attendance();
317             break;
318
319         case 9:
320             System.out.println("Thank You!");
321             break;
322
323         default:
324             System.out.println("Invalid Choice!");
325     }
326
327     } while (choice != 9);
328 }
329 }
330

```

19. Appendix B — References

- Oracle. The Java® Tutorials: Classes and Objects. docs.oracle.com/javase/tutorial/java/javaOO/.
- Oracle. Java Platform, Standard Edition API Specification: `java.util.ArrayList`. docs.oracle.com/javase/8/docs/api/java/util/ArrayList.html.
- Oracle. Java Platform, Standard Edition API Specification: `java.util.Scanner`. docs.oracle.com/javase/8/docs/api/java/util/Scanner.html.
- Oracle. The Java® Tutorials: The switch Statement. docs.oracle.com/javase/tutorial/java/nutsandbolts/switch.html.
- Oracle. The Java® Tutorials: The while and do-while Statements. docs.oracle.com/javase/tutorial/java/nutsandbolts/while.html.
- Gosling, J., Joy, B., Steele, G., Bracha, G., Buckley, A. The Java Language Specification. Oracle/Addison-Wesley.
- Course/departamental guidelines on academic attendance eligibility thresholds (institution-specific, commonly 75%), referenced here as the basis for the `attendance()` eligibility check.